Digital Systems

Lecture 2

State and Instructions

Mark van der Wilk

Hilary Term 2024

Department of
COMPUTER
SCIENCE

Following the course of past years by Mike Spivey

Today

How do instructions alter the CPU state?

- Understand the effect of instructions on the state of the CPU.
- Worked example of executing an instruction.

Today

? How do instructions alter the CPU state?

- Understand the effect of instructions on the state of the CPU.
- Worked example of executing an instruction.

? How are instructions encoded electronically?

And how does this affect programming the machine?

Today

How do instructions alter the CPU state?

- Understand the effect of instructions on the state of the CPU.
- Worked example of executing an instruction.

How are instructions encoded electronically?

And how does this affect programming the machine?

Decode machine code

I.e. how to take machine code, and determine what it will do.

Today

How do instructions alter the CPU state?

- Understand the effect of instructions on the state of the CPU.
- Worked example of executing an instruction.

How are instructions encoded electronically?

And how does this affect programming the machine?

Decode machine code

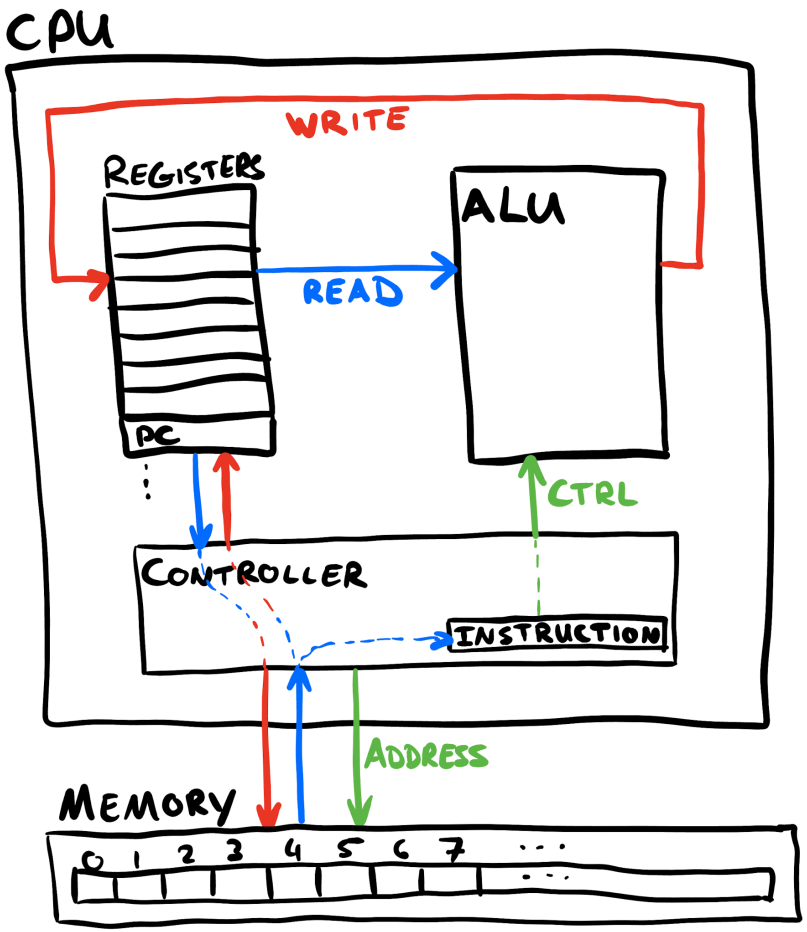
I.e. how to take machine code, and determine what it will do.

? How can we make parts of a program reusable?

How do subroutines work?

Operation

1. Controller fetches an instruction from memory, at location in Program Counter (PC)
2. Decode instruction
   RISC, so instructions are:
   - Load / Store
     (register ↔ memory)
   - Arithmetic / Logic
     (register ↔ register)
3. Execute instruction
4. Increment pc by 1 instruction, and repeat



Implementation

We will discuss electronics more later, but for now:

- All state is represented by an electrical voltage.

Implementation

We will discuss electronics more later, but for now:

- All state is represented by an electrical voltage.
- Voltage is either “high” or “low”.

Implementation

We will discuss electronics more later, but for now:

- All state is represented by an electrical voltage.
- Voltage is either “high” or “low”.
- Represents binary encoding of numbers.

Implementation

We will discuss electronics more later, but for now:

- All state is represented by an electrical voltage.
- Voltage is either “high” or “low”.
- Represents binary encoding of numbers.

We talk about “state”, and “registers”, and all that, but it’s important to keep in mind that…

Implementation

We will discuss electronics more later, but for now:

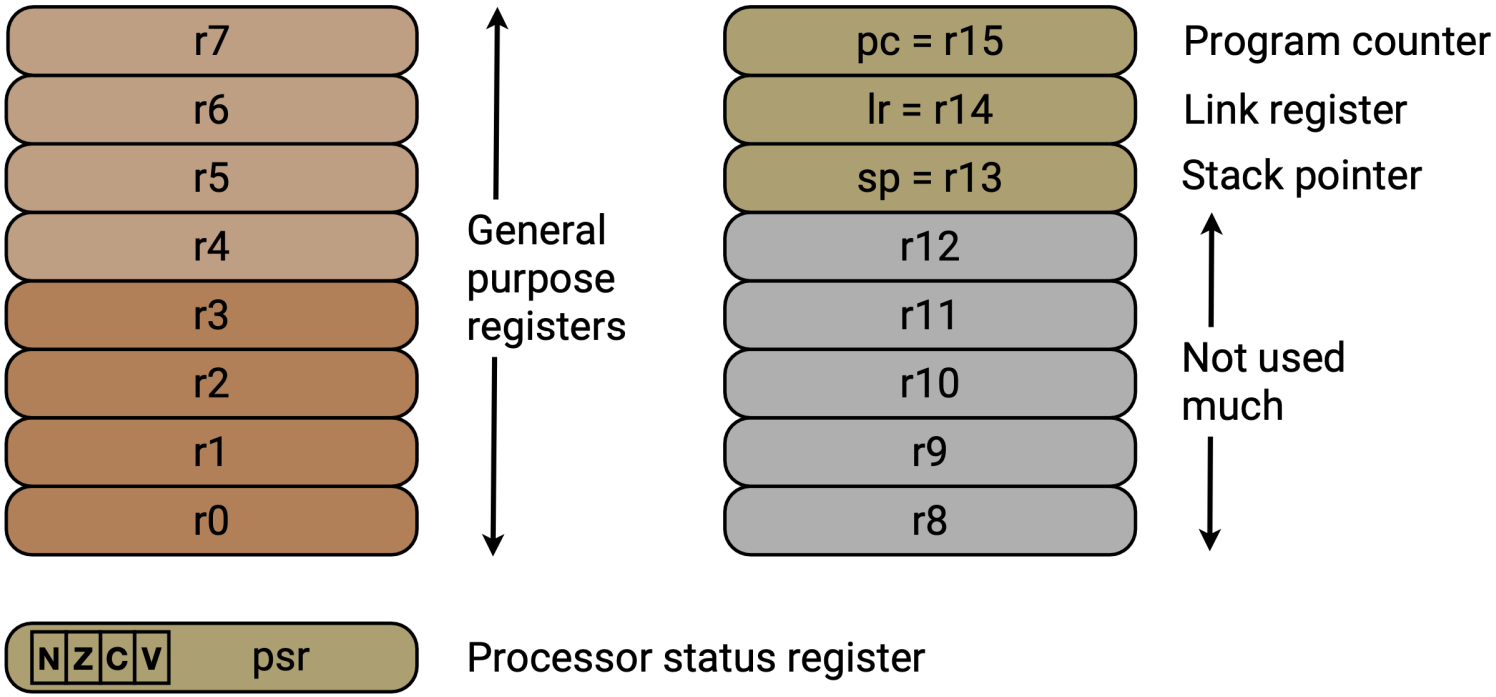
- All state is represented by an electrical voltage.
- Voltage is either “high” or “low”.
- Represents binary encoding of numbers.

We talk about “state”, and “registers”, and all that, but it’s important to keep in mind that…

> **What you are about to learn is *physically real***
>
> If you could open up the chip, and attach an LED circuit to the wires, you would literally see them light up according to binary.

Registers: A Closer Look

For most programming purposes, the state of the registers, is the state that we manipulate with instructions.



Registers: Purpose

All Registers are 32 bits “wide"

- `r0` - `r7` : General purpose registers. Can be used for any value used in any calculation.

Registers: Purpose

All Registers are 32 bits “wide"

- `r0 - r7` : General purpose registers. Can be used for any value used in any calculation.
- `r0 - r3` : Handled differently during subroutine calls.

Registers: Purpose

All Registers are 32 bits “wide"

- `r0 - r7` : General purpose registers. Can be used for any value used in any calculation.
- `r0 - r3` : Handled differently during subroutine calls.
- `r8 - r12` : Can be used for any value, but the instruction set makes it hard to access them, so not as “general purpose” as `r0 - r7`.

Registers: Purpose

All Registers are 32 bits “wide"

- `r0 - r7` : General purpose registers. Can be used for any value used in any calculation.
- `r0 - r3` : Handled differently during subroutine calls.
- `r8 - r12` : Can be used for any value, but the instruction set makes it hard to access them, so not as “general purpose” as `r0 - r7`.
- `pc` : Program counter. Stores memory address of next instruction.

Registers: Purpose

All Registers are 32 bits “wide"

- `r0 - r7` : General purpose registers. Can be used for any value used in any calculation.
- `r0 - r3` : Handled differently during subroutine calls.
- `r8 - r12` : Can be used for any value, but the instruction set makes it hard to access them, so not as “general purpose” as `r0 - r7`.
- `pc` : Program counter. Stores memory address of next instruction.
- `lr` : Link register. Stores address where current subroutine returns.

Registers: Purpose

All Registers are 32 bits “wide"

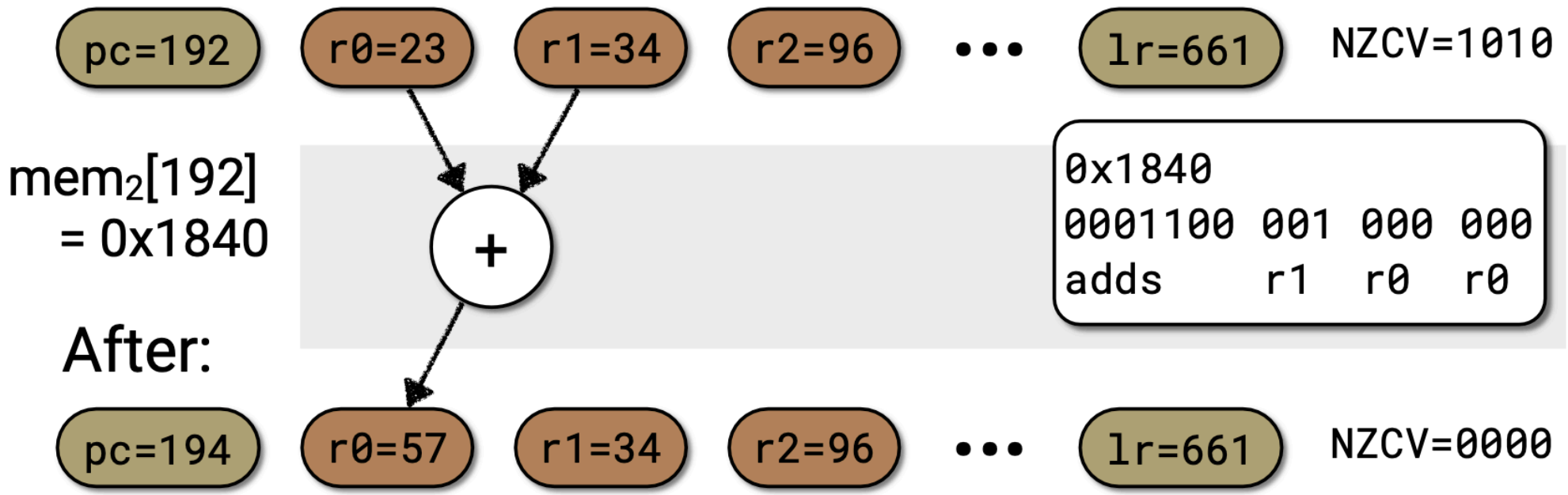
- `r0 - r7` : General purpose registers. Can be used for any value used in any calculation.
- `r0 - r3` : Handled differently during subroutine calls.
- `r8 - r12` : Can be used for any value, but the instruction set makes it hard to access them, so not as “general purpose” as `r0 - r7`.
- `pc` : Program counter. Stores memory address of next instruction.
- `lr` : Link register. Stores address where current subroutine returns.
- `sp` : Stack Pointer.

Registers: Purpose

All Registers are 32 bits “wide"

- `r0 - r7` : General purpose registers. Can be used for any value used in any calculation.
- `r0 - r3` : Handled differently during subroutine calls.
- `r8 - r12` : Can be used for any value, but the instruction set makes it hard to access them, so not as “general purpose” as `r0 - r7`.
- `pc` : Program counter. Stores memory address of next instruction.
- `lr` : Link register. Stores address where current subroutine returns.
- `sp` : Stack Pointer.
- `psr`: Processor Status Register. Different bits have very specific meanings. Some instructions write bits, some read bits.

The Effect of an Instruction



- Hex is convenient: Each hex digit corresponds to 4 bits.
- 0x1840 = 0b0001100001000000
- In assembly language: `adds r0, r0, r1`.

Status Bits

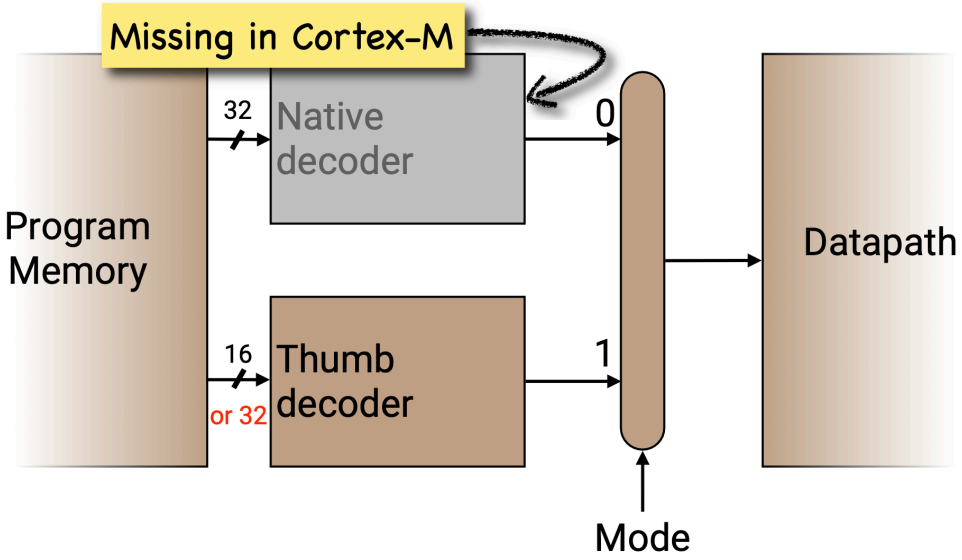
The status bits `N`, `Z`, `C`, `V` are in `psr`. The instruction `adds` is does “add"ition, and sets the “s"tatus bits.

- `N`: Set if the result is negative.
- `Z`: Set if the result is zero.
- `C`: Look at later.
- `V`: Look at later.

Program Counter

There are multiple encodings for ARM instructions. Big chips can choose.

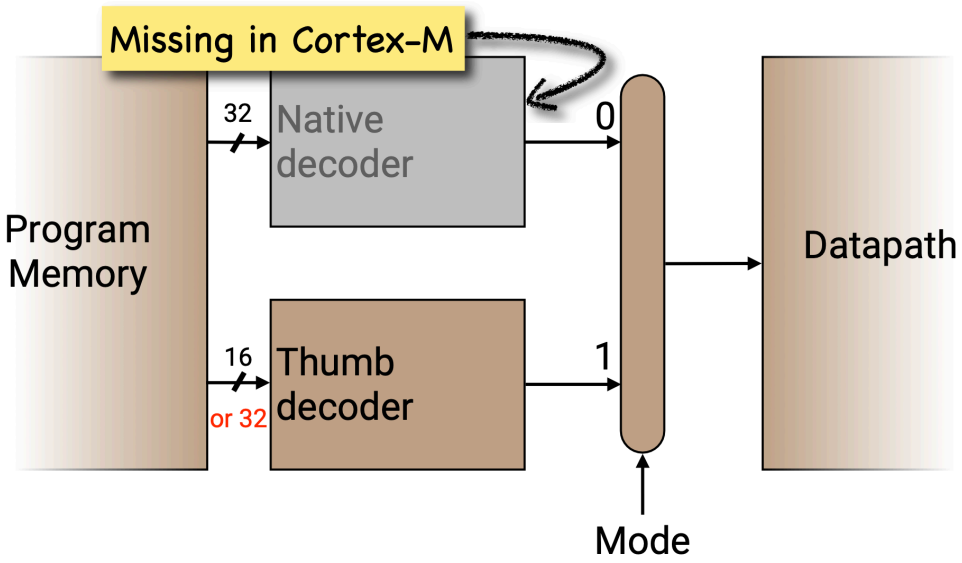
- "Native" ARM instructions (32 bits)



Program Counter

There are multiple encodings for ARM instructions. Big chips can choose.

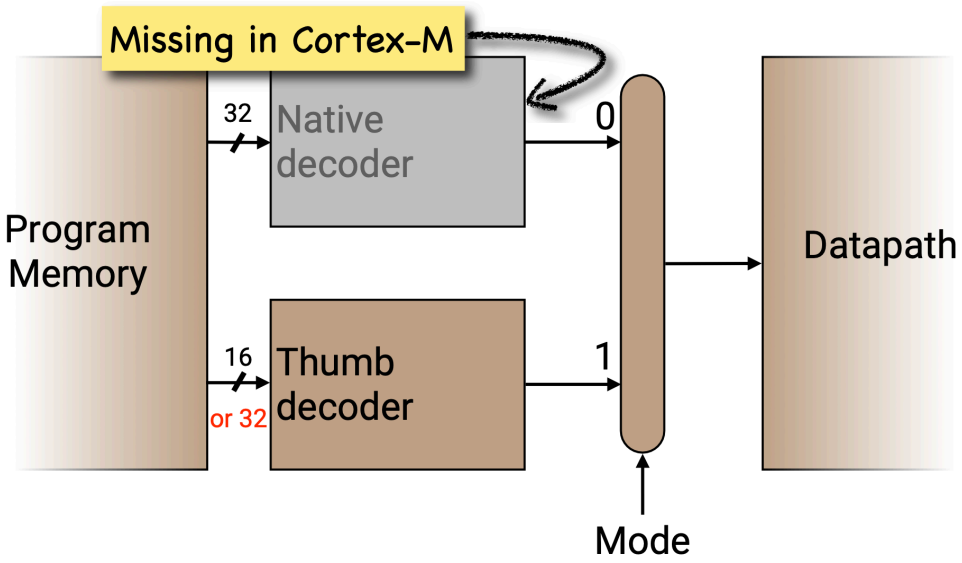
- "Native" ARM instructions (32 bits)
- "Thumb" instructions (16 bits)



Program Counter

There are multiple encodings for ARM instructions. Big chips can choose.

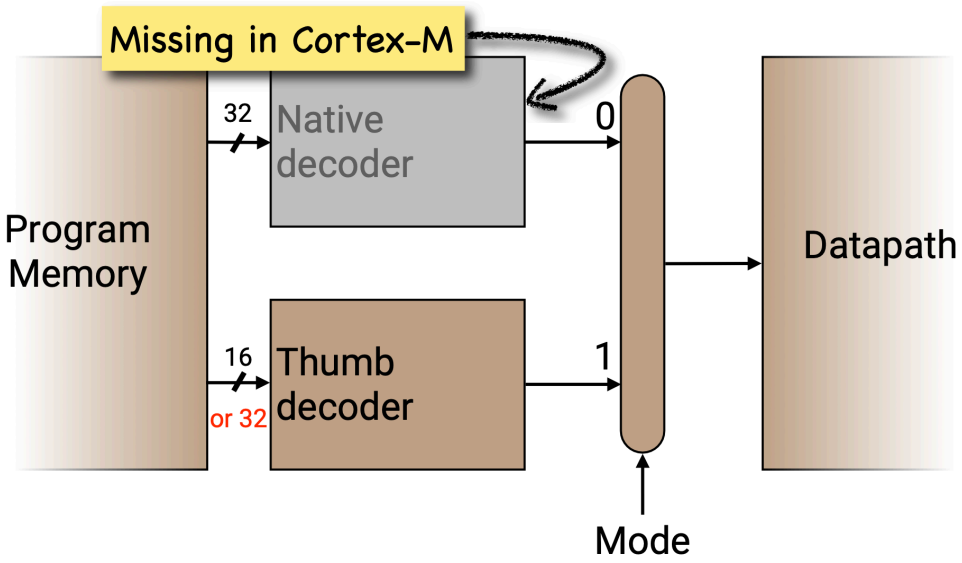
- "Native" ARM instructions (32 bits)
- "Thumb" instructions (16 bits)
- Thumb-2 extensions (mixture of 16- and 32 bit)



Program Counter

There are multiple encodings for ARM instructions. Big chips can choose.

- "Native" ARM instructions (32 bits)
- "Thumb" instructions (16 bits)
- Thumb-2 extensions (mixture of 16- and 32 bit)



Cortex-M0 only has Thumb instructions, but Cortex-M4 uses ThumbV2.

Assembly Language

- First level of abstraction up from machine code.
- Very close correspondence between assembly and machine code.
- Basically machine code in human-readable form, with tiny bits of "syntactic sugar".
- Program that translates from assembly to machine code: *Assembler*.
- Given close correspondence, assemblers are simple programs, particularly when compared to compilers.

Assembly Language has… “Quirks”

Consider the instruction

- `adds r0, r1, #3`

Assembly Language has… “Quirks”

Consider the instruction

- `adds r0, r1, #3`
- `adds r0, r0, #10`

Assembly Language has… “Quirks”

Consider the instruction

- `adds r0, r1, #3`
- `adds r0, r0, #10`
- `adds r0, r1, #10`

Assembly Language has… “Quirks”

Consider the instruction

- `adds r0, r1, #3`
- `adds r0, r0, #10`
- `adds r0, r1, #10`

Assembly Language has… “Quirks”

Consider the instruction

- `adds r0, r1, #3`
- `adds r0, r0, #10`
- `adds r0, r1, #10`

Very annoying!

Assembly Language has… “Quirks”

Consider the instruction

- `adds r0, r1, #3`
- `adds r0, r0, #10`
- `adds r0, r1, #10`

Very annoying!

- Exam won’t test whether you can memorise these rules.

Assembly Language has… “Quirks”

Consider the instruction

- `adds r0, r1, #3`
- `adds r0, r0, #10`
- `adds r0, r1, #10`

Very annoying!

- Exam won’t test whether you can memorise these rules.
- But we do want to understand why this happens!

Decoding machine code

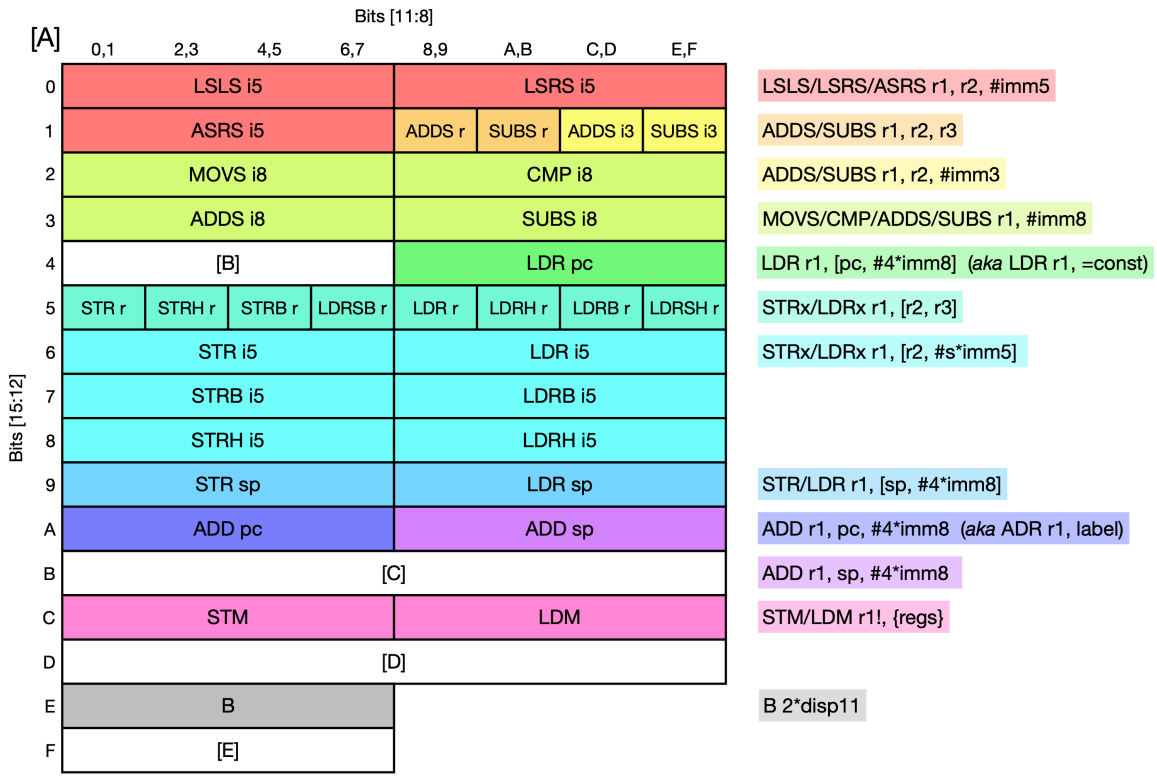
I.e. how to take machine code, and determine what assembly instruction it maps to.

- As an assembly level programmer, no need to know this.
- … unless you want to understand *why*
- But as a *machine code* programmer, you definitely do!
  - Or as a designer of the underlying logic circuits!
  - Or if you want to *write* an assembler.

Rainbow Chart

In Moodle you can find `rainbow-chart.pdf`.

- Nice summary of how binary patterns map to instructions.
- List of all instructions used in the course.
- Will be included in the exam.



0x1840 = 0b0001 100 001 000 000

adds r0, r0, r1

Consequences of Encoding

We know know *why*

Consequences of Encoding

We know know *why*

- Only the lower 8 registers can be accessed by most instructions.
  - Allocating 3 bits to indicate the register only gives 8 options!

Consequences of Encoding

We know know *why*

- Only the lower 8 registers can be accessed by most instructions.
  - Allocating 3 bits to indicate the register only gives 8 options!
- The assembler language `adds r0, r1, #10` is illegal.
  - We only have 3 bits to allocate to the constant, giving only 8 options!

The Next Instruction

Now the `pc` points to 194 in memory, and contains the instruction

$$0x4770 = 0b01000111\ 0\ 1110\ 000$$

The Next Instruction

Now the pc points to 194 in memory, and contains the instruction

$$0x4770 = 0b01000111\ 0\ 1110\ 000$$

which decodes to:

[B]

Bits [15:8]	Bits [7:4] 0,1,2,3	4,5,6,7	8,9,A,B	C,D,E,F	
40	ANDS	EORS	LSLS	LSRS	op r1, r2
41	ASRS	ADCS	SBCS	RORS	
42	TST	NEGS	CMP	CMN	
43	ORRS	MULS	BICS	MVNS	
44	ADD				ADD/CMP/MOV r/h1, r/h2
45	CMP				
46	MOV				
47	BX		BLX		BX/BLX r/h1

The Next Instruction

Now the pc points to 194 in memory, and contains the instruction

$$0x4770 = 0b01000111\ 0\ 1110\ 000$$

which decodes to: bx.

[B]

Bits [15:8] \ Bits [7:4]	0,1,2,3	4,5,6,7	8,9,A,B	C,D,E,F	
40	ANDS	EORS	LSLS	LSRS	op r1, r2
41	ASRS	ADCS	SBCS	RORS	
42	TST	NEGS	CMP	CMN	
43	ORRS	MULS	BICS	MVNS	
44	ADD				ADD/CMP/MOV r/h1, r/h2
45	CMP				
46	MOV				
47	BX		BLX		BX/BLX r/h1

The Next Instruction

Now the `pc` points to 194 in memory, and contains the instruction

$$0x4770 = 0b01000111\ 0\ 1110\ 000$$

which decodes to: `bx`.

Looking at the instruction reference:

Branch to reg	`bx` *rm*	*pc* := R*m*	High regs allowed
Branch reg & link	`blx` *rm*	*lr* := *next*; *pc* := R*m*	

[B]

Bits [15:8] \ Bits [7:4]	0,1,2,3	4,5,6,7	8,9,A,B	C,D,E,F	
40	ANDS	EORS	LSLS	LSRS	op r1, r2
41	ASRS	ADCS	SBCS	RORS	
42	TST	NEGS	CMP	CMN	
43	ORRS	MULS	BICS	MVNS	
44	ADD				ADD/CMP/MOV r/h1, r/h2
45	CMP				
46	MOV				
47	BX		BLX		BX/BLX r/h1

The Next Instruction

Now the `pc` points to 194 in memory, and contains the instruction

0x4770 = 0b01000111 0 1110 000

which decodes to: `bx`.

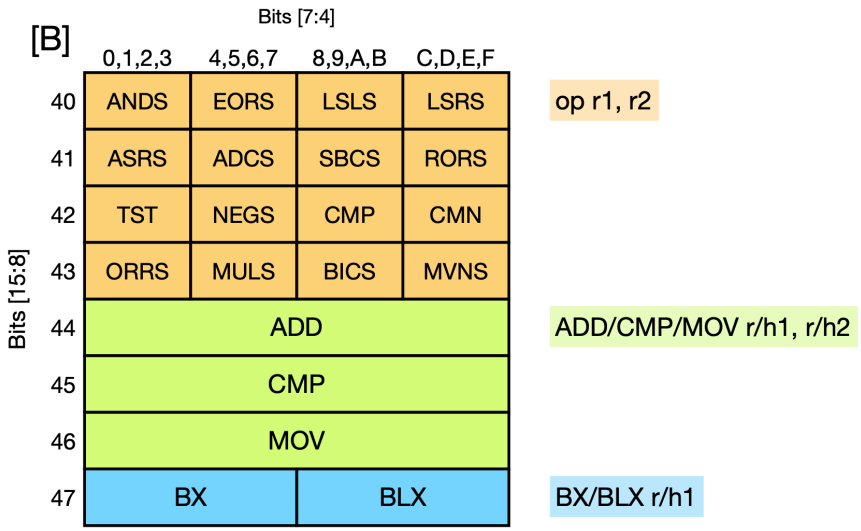
Looking at the instruction reference:

Branch to reg	`bx rm`	*pc* := R*m*	High regs allowed
Branch reg & link	`blx rm`	*lr* := *next*; *pc* := R*m*	

Regs encoded in following bits

`0b1110` = 14, and `r14` is `lr`, so:

`bx` lr



Datasheets: How we know how regs are encoded

This behaviour is not mysterious. It was *designed* by humans. Humans want others to understand. So they write documentation / datasheets. ARM publishes all this in easy-to-search formats!

So if you:

Datasheets: How we know how regs are encoded

This behaviour is not mysterious. It was *designed* by humans. Humans want others to understand. So they write documentation / datasheets. ARM publishes all this in easy-to-search formats!

So if you:

- go to **https://spivey.oriel.ox.ac.uk/digisys/The_BBC_micro:bit**,

Datasheets: How we know how regs are encoded

This behaviour is not mysterious. It was *designed* by humans. Humans want others to understand. So they write documentation / datasheets. ARM publishes all this in easy-to-search formats!

So if you:

- go to **https://spivey.oriel.ox.ac.uk/digisys/The_BBC_micro:bit**,
- click on ARM "architecture reference manual",

Datasheets: How we know how regs are encoded

This behaviour is not mysterious. It was *designed* by humans. Humans want others to understand. So they write documentation / datasheets. ARM publishes all this in easy-to-search formats!

So if you:

- go to **https://spivey.oriel.ox.ac.uk/digisys/The_BBC_micro:bit**,
- click on ARM "architecture reference manual",
- look at the contents, and find "A5.1 Thumb instruction set encoding",

Datasheets: How we know how regs are encoded

This behaviour is not mysterious. It was *designed* by humans. Humans want others to understand. So they write documentation / datasheets. ARM publishes all this in easy-to-search formats!

So if you:

- go to **https://spivey.oriel.ox.ac.uk/digisys/The_BBC_micro:bit**,
- click on ARM "architecture reference manual",
- look at the contents, and find "A5.1 Thumb instruction set encoding",

you will find all information that the rainbow chart was made from.

Datasheets: How we know how regs are encoded

This behaviour is not mysterious. It was *designed* by humans. Humans want others to understand. So they write documentation / datasheets. ARM publishes all this in easy-to-search formats!

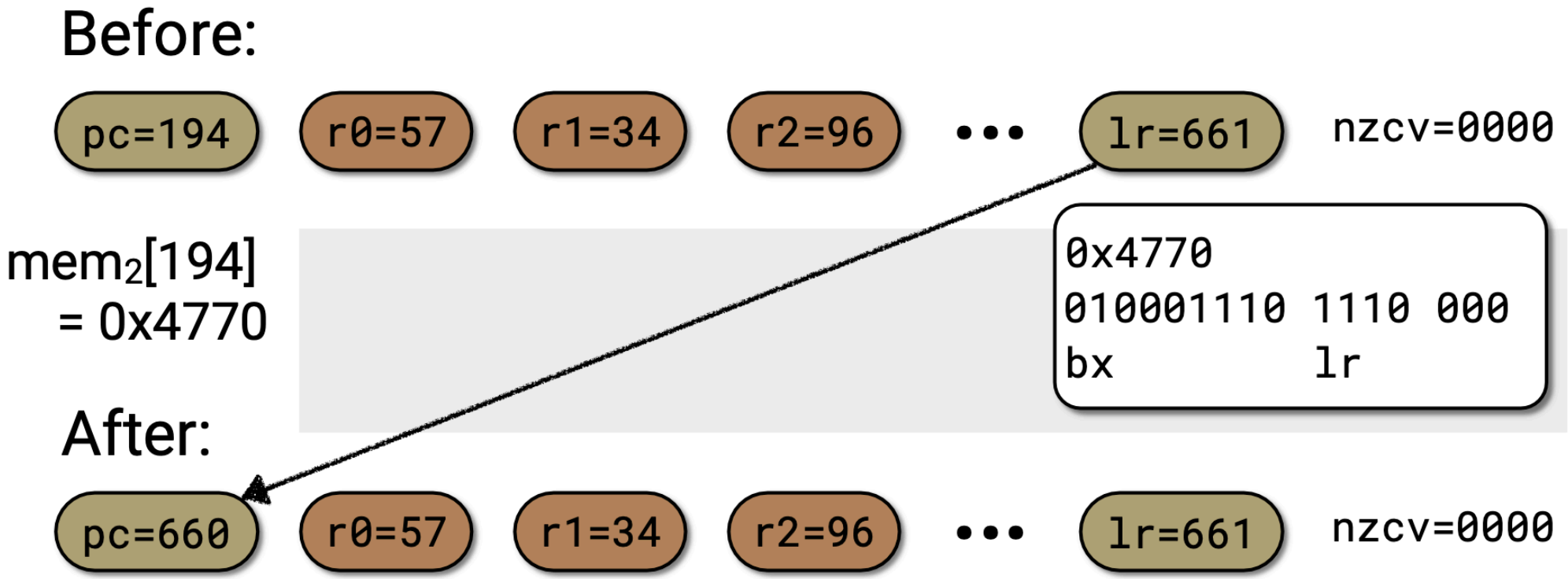
So if you:

- go to **https://spivey.oriel.ox.ac.uk/digisys/The_BBC_micro:bit**,
- click on ARM "architecture reference manual",
- look at the contents, and find "A5.1 Thumb instruction set encoding",

you will find all information that the rainbow chart was made from.

Follow links further, and you will find A6.7.15 for `bx`.

The Effect of `bx lr`



Subroutines

How can we make parts of a program reusable?

Subroutines

? How can we make parts of a program reusable?

Store a re-usable sequence of instructions, starting at a known memory location (e.g. 660). Then run program:

Subroutines

? How can we make parts of a program reusable?

Store a re-usable sequence of instructions, starting at a known memory location (e.g. 660). Then run program:

- Instr: Store location of next instruction in `lr`.

Subroutines

? How can we make parts of a program reusable?

Store a re-usable sequence of instructions, starting at a known memory location (e.g. 660). Then run program:

- Instr: Store location of next instruction in `lr`.
- Instr: Change the `pc` to known subroutine location.
  - Run multiple instructions in subroutine.
  - Change the `pc` back to `lr`.

Subroutines

? How can we make parts of a program reusable?

Store a re-usable sequence of instructions, starting at a known memory location (e.g. 660). Then run program:

- Instr: Store location of next instruction in `lr`.
- Instr: Change the `pc` to known subroutine location.
  - Run multiple instructions in subroutine.
  - Change the `pc` back to `lr`.
- Continue program.

Subroutines

? How can we make parts of a program reusable?

Store a re-usable sequence of instructions, starting at a known memory location (e.g. 660). Then run program:

- Instr: Store location of next instruction in `lr`.
- Instr: Change the `pc` to known subroutine location.
  - Run multiple instructions in subroutine.
  - Change the `pc` back to `lr`.
- Continue program.

We saw how

Subroutines

? How can we make parts of a program reusable?

Store a re-usable sequence of instructions, starting at a known memory location (e.g. 660). Then run program:

- Instr: Store location of next instruction in `lr`.
- Instr: Change the `pc` to known subroutine location.
  - Run multiple instructions in subroutine.
  - Change the `pc` back to `lr`.
- Continue program.

We saw how `bx lr`

Subroutines

? How can we make parts of a program reusable?

Store a re-usable sequence of instructions, starting at a known memory location (e.g. 660). Then run program:

- Instr: Store location of next instruction in `lr`.
- Instr: Change the `pc` to known subroutine location.
  - Run multiple instructions in subroutine.
  - Change the `pc` back to `lr`.
- Continue program.

We saw how `bx lr` allows the program to continue.

Subroutines

⚠ **Register values remain the same between subroutines!**

- When jumping into a subroutine, the registers may be in use by the caller.
- Subroutine needs to use registers as well!

⇒ We need conventions on how to use registers to prevent overwriting data that is in use, and how to communicate results of subroutine.

Lab Exercise One

- Practice writing assembly language.
- Your assembly language will be in a subroutine.
- Called from a bigger C program that handles I/O.

Conclusion

How do instructions alter the CPU state?

- Understand the effect of instructions on the state of the CPU.
- Worked example of executing an instruction.

How are instructions encoded electronically?

And how does this affect programming the machine?

Decode machine code

I.e. how to take machine code, and determine what it will do.

? How can we make parts of a program reusable?

How do subroutines work?